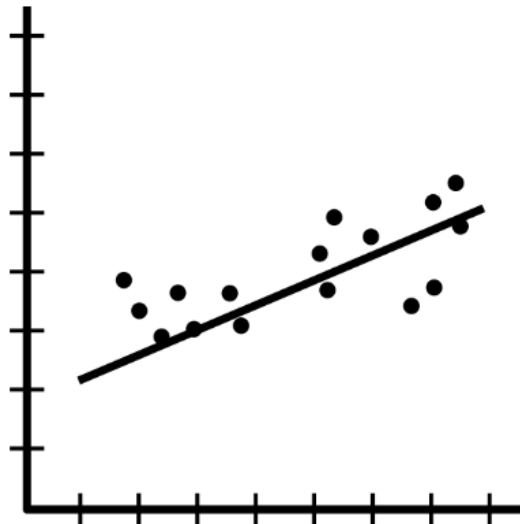


REGRESIÓN LINEAL

Introducción.....	2
¿Qué es un modelo predictivo?.....	2
Tipos de modelos predictivos.....	2
¿Qué es la regresión lineal?.....	3
Tipos de regresiones lineales.....	3
Enfoque matemático.....	4
Implementación en Python.....	5
Regresión simple.....	5
Carga de datos.....	5
Preparación de los datos.....	6
Entrenamiento del modelo.....	7
Interpretación de resultados.....	8
Coeficientes del modelo.....	8
Bondad del ajuste.....	8
Residuos.....	9
Predicciones.....	10
Regresión múltiple.....	11
Carga de datos.....	11
Preparación de los datos.....	12
Entrenamiento del modelo.....	13
Interpretación de resultados.....	14
Coeficientes del modelo.....	14
Bondad del ajuste.....	15
Residuos.....	15
Predicciones.....	16
Limitaciones y supuestos de las regresiones.....	17



Introducción

¿Qué es un modelo predictivo?

Un modelo predictivo es una herramienta matemática o estadística que utiliza datos históricos para predecir resultados futuros o desconocidos. A través de técnicas de aprendizaje automático, análisis de patrones y Big Data, estos modelos son capaces de identificar tendencias y relaciones en grandes volúmenes de datos, lo que permite realizar predicciones basadas en información previa. Los modelos predictivos se utilizan ampliamente en diversos sectores, como negocios, medicina, finanzas, marketing, redes sociales, y política, entre otros, y juegan un papel fundamental en la toma de decisiones informadas, optimizando procesos y anticipando comportamientos o eventos futuros.

Tipos de modelos predictivos

Modelo	Uso	Ejemplos
de regresión	Permiten predecir valores continuos o probabilidades.	Regresión lineal Regresión logística
de clasificación	Permiten predecir la pertenencia a una clase. Los resultados del modelo son binarios, o un sí o un no (en forma de 0 y 1) con su grado de probabilidad.	Árboles de decisión SVM (máquina de vector de soporte) KNN (vecinos más cercanos) regresión logística
de ensamble	Crea un nuevo modelo entrenando varios modelos similares y combinando los resultados para mejorar la precisión, reducir la varianza y los sesgos e identificar el mejor modelo para usar con nuevos datos.	Bosque aleatorio Refuerzo de gradiente
redes neuronales	Técnica de reconocimiento de patrones que imita las neuronas del cerebro humano ya que es capaz de modelar relaciones extremadamente complejas y suele utilizarse cuando no se conoce la naturaleza exacta de la relación entre los valores de entrada y los de salida.	Deep Learning

de clustering	Separa los datos en diferentes categorías basadas en características similares y después utiliza los datos de cada grupo para determinar los resultados a gran escala para cada clúster.	K-means DBSCAN
de series temporales	Un modelo de series temporales utiliza secuencias de tiempo como factores de entrada en un conjunto de datos. Considera las tendencias pasadas y los puntos de datos de una secuencia de tiempo específica para predecir tendencias o sucesos a lo largo de otro período de tiempo específico.	ARIMA Prophet Modelos de suavizado exponencial

¿Qué es la regresión lineal?

La regresión lineal es una técnica estadística ampliamente utilizada que permite predecir el comportamiento de una variable numérica (dependiente) a partir de una o más variables (independientes), mediante el ajuste de una ecuación lineal. Su simplicidad e interpretabilidad la han consolidado como una herramienta esencial en análisis predictivo.

En el ámbito empresarial y científico, la regresión lineal no solo predice valores futuros, sino que cuantifica el impacto de cada variable independiente sobre el resultado. Por ejemplo, permite responder preguntas como: ¿Cuánto aumenta el precio de una vivienda por cada metro cuadrado adicional? o ¿Qué porcentaje de las ventas depende del presupuesto en marketing?. Estas capacidades, combinadas con su facilidad de implementación, la convierten en la piedra angular para la toma de decisiones basadas en evidencia, incluso en escenarios con datos limitados.

Tipos de regresiones lineales

Existen dos tipos principales de regresión lineal:

- **Simple:** se analiza la relación entre una variable dependiente y una variable independiente. Por ejemplo, podríamos estudiar cómo la cantidad de horas de estudio (variable independiente) se relaciona con el puntaje obtenido en un examen (variable dependiente).
- **Múltiple:** se estudia la relación entre una variable dependiente y dos o más variables independientes; lo cual permite tener en cuenta múltiples factores que podrían influir en la variable dependiente. Por ejemplo, predecir el precio

de una casa (variable dependiente) según el área construida, el número de habitaciones y la distancia al centro de la ciudad (variables independientes).

Enfoque matemático

Recordemos la fórmula de la recta, que es la base de la regresión lineal:

$$y = mx + b$$

El diagrama muestra la ecuación $y = mx + b$ con cuatro flechas verticales que apuntan a los términos de la ecuación desde etiquetas debajo:

- Una flecha naranja apunta desde "Variable dependiente" hasta el y .
- Una flecha azul apunta desde "Pendiente" hasta el m .
- Una flecha verde apunta desde "Variable independiente" hasta el x .
- Una flecha magenta apunta desde "Punto de corte eje Y (cuando $x=0$)" hasta el b .

La regresión lineal simple se define mediante la función lineal:

$$Y = \beta_0 X + \beta_1 + \varepsilon$$

En la regresión lineal múltiple, la función cambia para incluir más factores:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \varepsilon$$

- ❖ β_0 es la ordenada en el origen, se corresponde con el valor de Y (variable respuesta) cuando todas las X (variables predictoras) son cero. Se conoce como intercepto
- ❖ β_j es el efecto que tiene cada variable predictora X_j sobre la variable respuesta (Y), manteniendo fijas el resto de variables. Se conocen como coeficientes de regresión.
- ❖ ε es el término de error que representa la diferencia entre el valor predicho y el real. Incluye factores no medidos o variabilidad aleatoria. **El objetivo de la regresión lineal** es encontrar la mejor recta de ajuste que minimice el error entre las predicciones del modelo y los valores reales.

En la mayoría de los casos, no conocemos los valores reales de β_0 (intercepto) y β_j (coeficientes de las variables predictoras), sino que solo tenemos estimaciones

teóricas. El proceso de ajustar el modelo consiste en calcular los valores de los coeficientes que mejor se ajusten a los datos disponibles. Para hacer esto, buscamos los coeficientes que maximizan la probabilidad de que esos valores sean los que realmente generaron los datos observados.

Uno de los métodos más comunes para ajustar un modelo de regresión es el método de Mínimos Cuadrados. Este método busca la recta (o plano, en el caso de la regresión múltiple) que minimiza la suma de los errores al cuadrado. Los errores son las diferencias entre los valores predichos por el modelo y los valores reales. Al minimizar la suma de los errores al cuadrado, logramos un modelo que, en promedio, tiene los menores errores posibles. (Si tienes curiosidad por ver un ejemplo a mano del método de mínimos cuadrados dejo este [enlace](#))

Implementación en Python

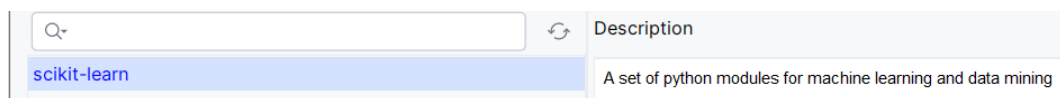
Regresión simple

Vamos a trabajar con un fichero que contiene datos de personas sobre sus años de experiencia laborales y sus ingresos anuales (`regresion_lineal_simple.csv`). La variable dependiente, es decir, la que vamos a predecir, es el salario y la independiente es la experiencia.

Las librerías que vamos a usar son las siguientes:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn import linear_model
```

Para importar **sklearn** tenemos que instalar el paquete **scikit-learn**



Carga de datos

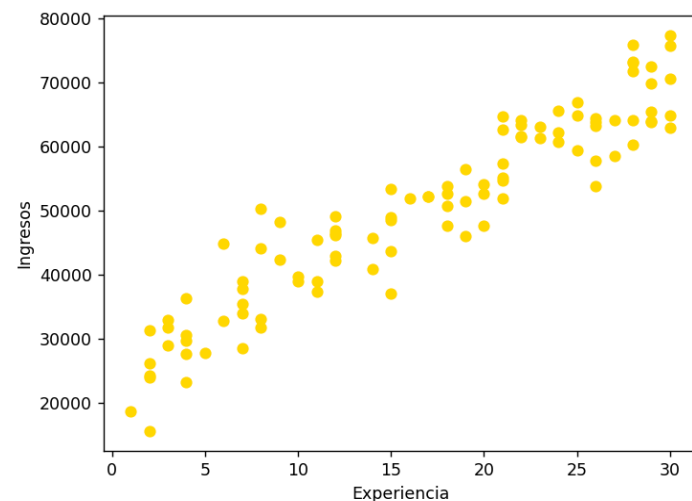
En este caso el fichero es un csv así que usaremos la función de pandas que nos permite leer este tipo de ficheros.

```
# Leemos los datos del fichero
datos = pd.read_csv("regresion_lineal_simple.csv",
encoding='latin1')
print(datos)
```

	Experiencia	Ingresos
0	7	38991.994707
1	20	54141.855824
2	29	63964.067099
3	15	53443.128632
4	11	45427.660622
..	...	...
95	29	65404.653198
96	18	53780.799152
97	26	63890.595628
98	12	49090.252189
99	2	26052.709878

```
# Mostramos el gráfico de dispersión para ver los datos
plt.ylabel("Ingresos")
plt.xlabel("Experiencia")
plt.scatter(datos["Experiencia"], datos["Ingresos"], color="gold")

plt.show()
```



Preparación de los datos

Como este ejemplo es de regresión simple, solo tenemos una variable independiente y es más sencilla la preparación de los datos. Posteriormente vamos a emplear la función **fit** para ajustar la regresión, es decir, entrenar el modelo para encontrar los coeficientes (β_0 , β_1) que mejor se adapten a los datos.

fit pide dos parámetros como argumentos: la X (o variables predictoras) y la Y (variable a predecir).

La X debe ser siempre una matriz 2D, incluso cuando solo tengamos una variable predictora como en este caso. La Y debe ser un vector 1D. Para obtener estos formatos se hará uso de **values**.

```
# Preparamos X (2D) e Y (1D) para el metodo fit de scikit-learn
x = datos["Experiencia"].values.reshape((-1, 1))
y = datos["Ingresos"].values
```

Entrenamiento del modelo

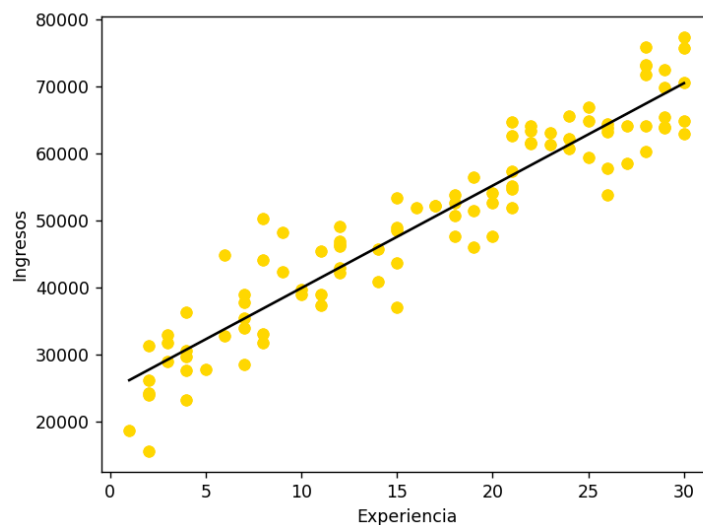
La función **LinearRegression()** inicializa un modelo de regresión lineal vacío y **fit** es la que entrena/ajusta el modelo a los datos.

Para mostrar la línea de regresión, necesitamos generar una serie de valores para el eje x (método **linspace**), luego usar el modelo de regresión para predecir los valores correspondientes en el eje y (método **predict**), y finalmente graficar esos puntos para visualizar la línea de regresión.

```
# Calculamos nuestra regresión lineal
regresion = linear_model.LinearRegression()
modelo = regresion.fit(x, y)

# Generamos valores para mostrar la recta de regresión
exp_rango = np.linspace(min(datos["Experiencia"]), max(datos["Experiencia"]), 100)
predicciones_rango = modelo.predict(exp_rango.reshape(-1, 1))

# Mostramos la recta de regresión
plt.scatter(datos["Experiencia"], datos["Ingresos"], color="gold")
plt.plot(exp_rango, predicciones_rango, color="black")
plt.xlabel("Experiencia")
plt.ylabel("Ingresos")
plt.show()
```



Interpretación de resultados

Coeficientes del modelo

- Intercepto (β_1): Es el valor esperado de ingresos cuando la experiencia es cero.
- Pendiente (β_0): Muestra el cambio en ingresos por cada año adicional de experiencia.

```
print("Intercepto (b o  $\beta_1$ )", modelo.intercept_)  
print("Pendiente (m o  $\beta_0$ )", modelo.coef_)  
  
Intercepto (b o  $\beta_1$ ) 24587.430704774157  
Pendiente (m o  $\beta_0$ ) [1531.11772861]
```

Con esto podríamos decir que una persona sin experiencia ganaría aproximadamente 24,587.43€ anuales y que por cada año de experiencia sus ingresos aumentarían 1,531.12€.

Bondad del ajuste

Como hemos visto, la regresión lineal calcula una ecuación que minimiza la distancia entre la línea ajustada y todos los puntos de los datos. En términos generales, un modelo se considera adecuado si las diferencias entre los valores observados y las predicciones del modelo son pequeñas.

El R-cuadrado (o coeficiente de determinación) es una medida estadística que indica qué tan bien se ajustan los datos a la línea de regresión. El R-cuadrado se expresa como un porcentaje, siempre en el rango de 0% a 100%, y refleja la cantidad de variabilidad en la variable dependiente que es explicada por el modelo lineal.

- Un valor de 0% indica que el modelo no explica nada de la variabilidad de los datos en torno a su media.
- Un valor de 100% indica que el modelo explica toda la variabilidad de los datos en torno a su media.

En general, cuanto mayor sea el R-cuadrado, mejor será el ajuste del modelo a los datos observados. Para obtener el R^2 vamos a usar el método **score**

```
#Calculamos  $R^2$   
r_cuadrado = modelo.score(x, y)
```



```
print(f"R²: {r_cuadrado:.2f}")
```

R²: 0.89

En este ejemplo el R-cuadrado es de 0.89. Indica que el 89% de la variación en los ingresos (variable dependiente) se explica por la experiencia (variable predictora). ¡Es un buen ajuste! Sin embargo, siempre es importante verificar los residuos para asegurarse de que el modelo no esté cometiendo errores sistemáticos.

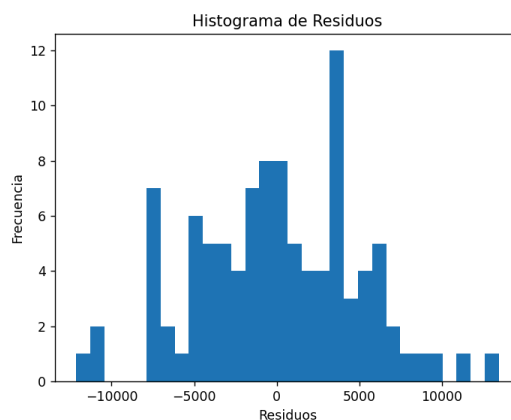
Residuos

Verificar los residuos es una parte importante del análisis de regresión porque nos permite evaluar la calidad del modelo y detectar posibles problemas, como patrones no explicados por el modelo. Los residuos son las diferencias entre los valores reales y los valores predichos por el modelo.

Un histograma de residuos nos ayuda a verificar si siguen una distribución normal, lo cual es esencial en muchos modelos de regresión. Idealmente, los residuos deben aproximarse a una distribución en forma de campana de Gauss para que las suposiciones del modelo sean válidas.

```
# Residuos
prediccion = modelo.predict(x)
residuos = y - prediccion

plt.hist(residuos, bins=30)
plt.xlabel('Residuos')
plt.ylabel('Frecuencia')
plt.title('Histograma de Residuos')
plt.show()
```



El histograma de residuos muestra que los residuos están distribuidos cerca de 0, lo que indica que el modelo de regresión no tiene errores sistemáticos. Sin embargo,

los picos en el histograma pueden sugerir que el modelo podría mejorar en algunos casos. En general, la distribución aleatoria de los residuos es una señal de que el modelo se ajusta razonablemente bien a los datos.

Predicciones

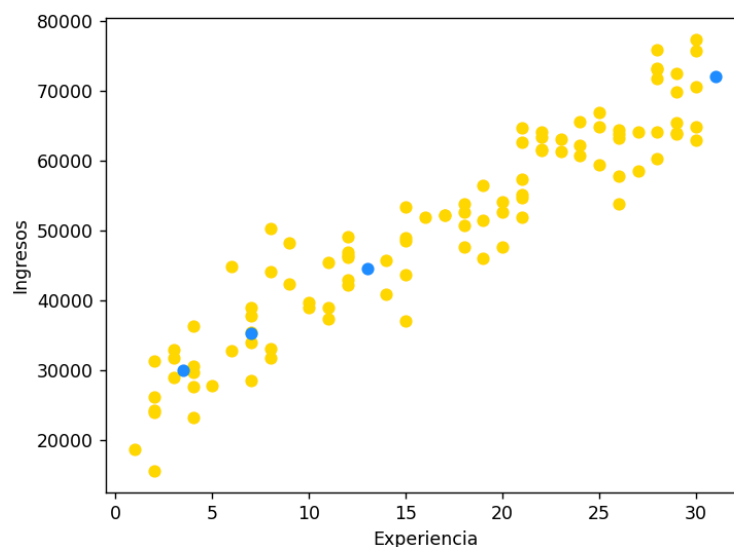
Con los modelos de regresión lineal, podemos hacer predicciones sobre la variable dependiente, en este caso el salario. Esto nos permite responder preguntas como "¿Cuánto cobrará una persona con X años de experiencia?". Vamos a ver la predicción que nos da para unos valores específicos:

```
# Ahora vamos a predecir el salario de personas con esta experiencia
entrada = [[3.5], [7], [13], [31]]
prediccion = modelo.predict(entrada)
print(prediccion)

[29946.34275492 35305.25480507 44491.96117675 72052.08029179]

#Mostramos la gráfica con las predicciones en azul
plt.scatter(datos["Experiencia"], datos["Ingresos"], color="gold")
plt.scatter(entrada, prediccion, color="dodgerblue")
plt.ylabel("Ingresos")
plt.xlabel("Experiencia")

plt.show()
```



Con nuestro modelo, podemos predecir que una persona con 3 años y medio de experiencia tendrá un sueldo cercano a los 30.000€ (29,946.34€), mientras que una persona con 13 años de experiencia recibirá un salario de 44,491.96€

Regresión múltiple

Vamos a seguir los mismos pasos que en el ejemplo anterior de regresión simple pero esta vez con una múltiple para ver las diferencias a la hora de trabajar con varias variables independientes. El ejemplo trata del precio de viviendas según el área en metros cuadrados y la distancia de la vivienda al centro del pueblo o ciudad (regresion_multiple.csv).

Carga de datos

En este caso el fichero vuelve a ser un csv así que usaremos la misma función que en el ejemplo previo para leer este tipo de ficheros. Para la representación vamos a usar una gráfica 3D porque tenemos tres parámetros, tenemos que importar mplot3d que nos da las herramientas para ello.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn import linear_model

# Leemos los datos del fichero
datos = pd.read_csv("regresion_multiple.csv", encoding='latin1')
print(datos)
```

	Precio	Area	Distancia_centro
0	121209.78	70	15.00
1	130026.21	50	15.00
2	176399.42	68	15.00
3	180240.31	43	15.00
4	186370.52	89	15.00
5	191458.60	75	15.00
6	203199.49	91	15.00
7	210644.47	78	15.00
8	225958.39	83	15.00
9	232818.82	94	15.00

```
# Dibujamos el gráfico de dispersión
x1 = datos['Area']
y1 = datos['Distancia_centro']
z1 = datos['Precio']

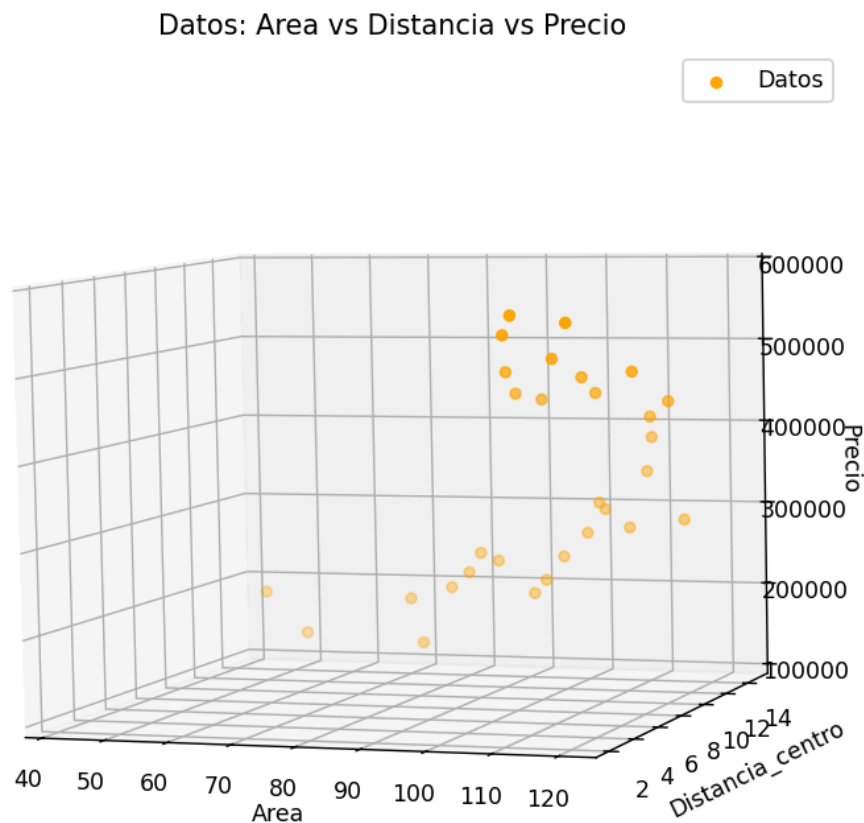
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
scatter = ax.scatter(x1, y1, z1, c='orange', label='Datos')
```

```

ax.set_xlabel('Area')
ax.set_ylabel('Distancia_centro')
ax.set_zlabel('Precio')
ax.set_title('Datos: Area vs Distancia vs Precio')

plt.legend()
plt.tight_layout()
plt.show()

```



Preparación de los datos

En este caso, estamos trabajando con más de una variable independiente o predictora. Por lo tanto, al realizar el ajuste de regresión, utilizaremos dos columnas como variables predictoras en lugar de solo una. Si en algún momento tenemos más de dos variables predictoras, simplemente debemos añadirlas a la variable x.

```

# Preparamos datos
x = datos[['Area', 'Distancia_centro']].values
y = datos['Precio'].values

```

Entrenamiento del modelo

En una regresión múltiple, al trabajar con más de una variable predictora, la relación entre las variables y la variable dependiente no se puede representar con una simple línea recta, como en la regresión lineal simple. En lugar de eso, la relación se muestra en un plano en un espacio tridimensional (o en dimensiones superiores si se tienen más variables predictoras). El modelo de regresión múltiple ajusta un plano que intenta minimizar el error entre los valores reales y las predicciones, similar a cómo la regresión simple ajusta una línea.

Para generar el plano de regresión en 3D, primero calculamos los coeficientes de la regresión usando el método **fit()**, el cual ajusta el modelo a los datos. Luego, para visualizar el plano, generamos una malla de puntos utilizando **meshgrid()** con los valores de las variables predictoras. Con esta malla de puntos, aplicamos la ecuación de la regresión para obtener los valores z (la predicción de la variable dependiente, en este caso, el precio). Finalmente, usamos **plot_surface()** para dibujar el plano en el gráfico 3D, mostrando cómo la combinación de las variables predictoras afecta al valor de la variable dependiente.

```
# Calculamos nuestra regresión lineal
regresion = linear_model.LinearRegression()
regresion.fit(x, y)

# Generamos valores para mostrar el plano de regresión
tam_rango = np.linspace(x1.min(), x1.max(), 50)
dist_rango = np.linspace(y1.min(), y1.max(), 50)
x_plano, y_plano = np.meshgrid(tam_rango, dist_rango)

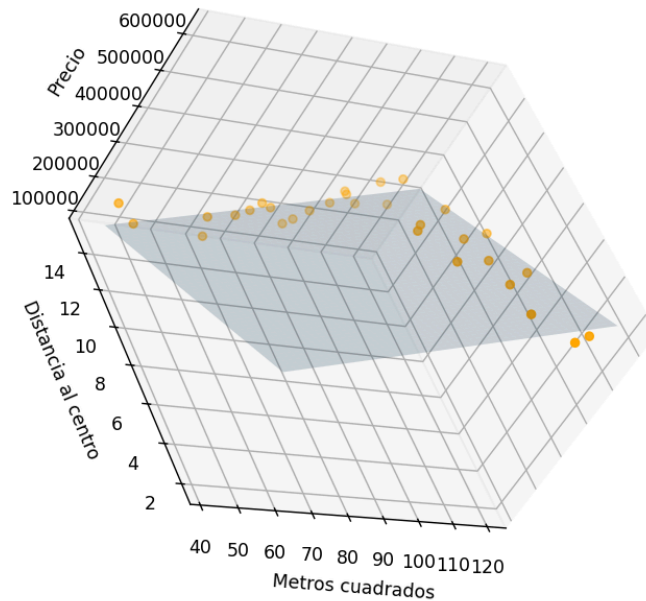
# Calcular los valores z del plano. Función  $\beta_0 + \beta_1x_1 + \beta_2x_2$ 
z_plano = regresion.intercept_ + regresion.coef_[0]*x_plano +
regresion.coef_[1]*y_plano

fig2 = plt.figure(figsize=(8, 6))
ax2 = fig2.add_subplot(111, projection="3d")

scatter = ax2.scatter(x1, y1, z1, c='orange', label='Datos')
ax2.plot_surface(x_plano, y_plano, z_plano, alpha=0.2)

ax2.set_xlabel("Metros cuadrados")
ax2.set_ylabel("Distancia al centro")
ax2.set_zlabel("Precio")

plt.show()
```



Interpretación de resultados

Coeficientes del modelo

Los coeficientes obtenidos en la regresión múltiple representan la relación entre cada variable independiente (predictora) y la variable dependiente (en este caso, el precio de la vivienda).

```
# Obtenemos los coeficientes de las variables independientes
print ('Coeficientes: ', regresion.coef_)

      Coeficientes: [ 2886.47420145 -22328.6410586 ]

# Obtenemos la intercepción
print ('Intercepción: ', regresion.intercept_)

      Intercepción: 326313.9737916034
```

El primer coeficiente (2886.47) está asociado al área en metros cuadrados de la vivienda, lo que significa que por cada metro cuadrado adicional, el precio de la vivienda aumentaría en 2,886.47€, manteniendo constante la distancia al centro. El segundo coeficiente (-22328.64) está relacionado con la distancia al centro de la vivienda, indicando que por cada kilómetro adicional de distancia al centro, el precio de la vivienda disminuye en 22,328.64€, manteniendo constante el área.

El intercepto (326,313.97€) representa el valor base del precio de la vivienda cuando ambos factores, área y distancia al centro, son cero. Puede interpretarse como el precio “teórico” para una vivienda “teórica” con un área de 0 m² y una distancia infinita al centro.

Bondad del ajuste

```
#Calculamos  $R^2$ 
r_cuadrado = regresion.score(x, y)
print(f" $R^2$ : {r_cuadrado:.2f}")

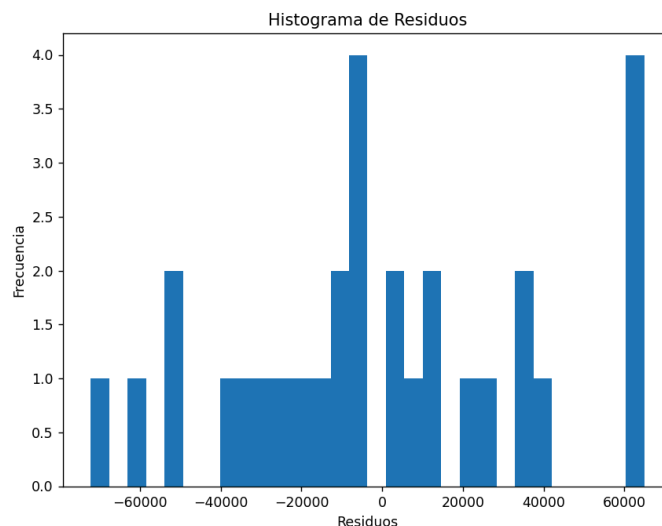
 $R^2$ : 0.93
```

Un R-cuadrado de 0.93 indica que el modelo de regresión lineal explica el 93% de la variabilidad en los precios de las viviendas con las variables predictoras (área y distancia al centro). Esto sugiere que el modelo tiene un ajuste excelente a los datos. Sin embargo, siempre es importante verificar los residuos para asegurarse de que el modelo no esté cometiendo errores sistemáticos.

Residuos

```
# Calculamos los residuos
predicciones = regresion.predict(x)
residuos = y - predicciones

histograma = plt.figure(figsize=(8, 6))
plt.hist(residuos, bins=30)
plt.xlabel('Residuos')
plt.ylabel('Frecuencia')
plt.title('Histograma de Residuos')
plt.show()
```



El histograma de residuos muestra que los residuos están distribuidos de manera bastante dispersa alrededor de 0, con algunos picos notables. Esto sugiere que el modelo de regresión tiene errores más grandes que el del ejemplo anterior en ciertos puntos. Los picos pueden indicar outliers o casos donde el modelo no se ajusta tan bien, lo que sugiere que podría haber margen de mejora.

Aunque el modelo tiene un ajuste del 93% (lo que indica que explica la mayor parte de la variabilidad en los datos), los residuos dispersos y los picos en el histograma sugieren que el modelo no está haciendo predicciones perfectas en todos los casos. Esto no significa que el modelo sea malo, pero no podemos fiarnos completamente de las predicciones, especialmente en los casos donde los residuos son grandes (es decir, donde las predicciones están muy lejos de los valores reales).

Predicciones

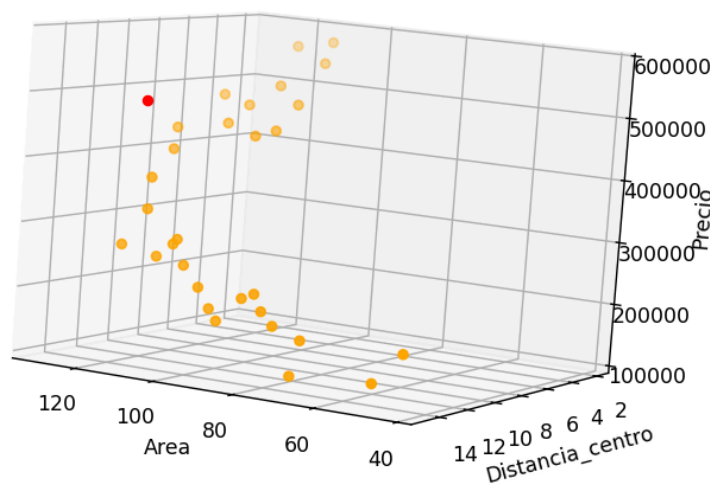
Con los modelos de regresión lineal, podemos hacer predicciones sobre la variable dependiente, en este caso el precio de la vivienda. Esto nos permite responder preguntas como "¿Cuánto costará una vivienda de 130 metros cuadrados situada a 10km del centro de la ciudad más cercana? Vamos a ver la predicción que nos da:

```
# Predecimos el precio de una vivienda de 130m2 a 10km del centro
entrada = [[130, 10]]
prediccion = regresion.predict(entrada)
print(prediccion)
```

[478269.20939365]

```
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
scatter = ax.scatter(x1, y1, z1, c='orange', label='Datos')
scatter = ax.scatter(130, 10, prediccion, c='red',
label='Prediccion')

ax.set_xlabel('Area')
ax.set_ylabel('Distancia_centro')
ax.set_zlabel('Precio')
plt.show()
```



Estima que el piso o la casa costará sobre los 478,269.21€.

Limitaciones y supuestos de las regresiones

Para finalizar, veremos resumidamente los supuestos básicos para que el modelo de regresión lineal funcione correctamente. Si alguno de estos no se cumple, el modelo podría no ser adecuado o necesitar ajustes:

- ☐ Linealidad: Se asume que la relación entre las variables predictoras y la variable dependiente es lineal.
- ☐ Independencia: Las observaciones deben ser independientes entre sí, es decir, el valor de una observación no debe depender de las demás.
- ☐ Homoscedasticidad: Los errores (diferencias entre valores predichos y reales) deben tener una distribución constante a lo largo de todas las observaciones.
- ☐ Normalidad de los residuos: Los residuos deben seguir una distribución normal para que las inferencias sean fiables.